

磁盘容量排序

题目描述

磁盘的容量单位常用的有 M，G，T 这三个等级，它们之间的换算关系为：

- $1T = 1024G$
- $1G = 1024M$

现在给定 n 块磁盘的容量，请对它们按从小到大的顺序进行 **稳定排序**。

例如给定5块盘的容量：

1T, 20M, 3G, 10G6T, 3M12G9M

排序后的结果为：

20M, 3G, 3M12G9M, 1T, 10G6T

注意单位可以重复出现，上述 3M12G9M 表示的容量即为：3M+12G+9M，和 12M12G 相等。

输入描述

输入第一行包含一个整数 n，表示磁盘的个数

- $2 \leq n \leq 100$

接下的 n 行，每行一个字符串（长度大于2，小于30），表示磁盘的容量，由一个或多个格式为mv的子串组成，其中 m 表示容量大小，v 表示容量单位，例如：20M，1T，30G，10G6T，3M12G9M。

- 磁盘容量 m 的范围为 1 到 1024 的正整数
- 容量单位 v 的范围只包含题目中提到的 M，G，T 三种，换算关系如题目描述

输出描述

输出 n 行，表示 n 块磁盘容量排序后的结果。

用例

输入	3 1G 2G 1024M
输出	1G 1024M 2G
说明	1G和1024M容量相等，稳定排序要求保留它们原来的相对位置，故1G在1024M之前。

输入	3 2G4M 3M2G 1T
输出	3M2G 2G4M 1T

说明	1T的容量大于2G4M，2G4M的容量大于3M2G。
----	----------------------------

题目解析

本题主要考察：自定义排序，稳定排序，字符串解析。

所谓稳定排序，即对于数组中等价的两个元素a，b来说，如果排序前，a 在 b 之前，那么排序后也要保证 a 在 b 之前。

比如题目自带的用例1中：1G 和 1024M 是等价的，但是输入顺序中 1G 在 1024M 之前，所以输出顺序中 1G 也应该在 1024M 之前。

我们常用的快速排序是不稳定的。而大部分编程语言的sort函数底层都是用的快速排序。

想要基于快速排序实现 稳定排序的话，一般的话，需要给数组元素带上原始索引位置信息，比如

```
disks = ["1G", "2G", "1024M"]
```

可以变为

```
disks = [{capacity: "1G", index: 0}, {capacity:"2G", index:1}, {capacity:"1024M", index:3}]
```

当两个元素的 capacity 相同时，则继续按照 index 升序。

但是，目前Java,JS,Python的sort函数在指定比较器后都能实现稳定排序。C++有stable_sort函数实现稳定排序。只有C语言需要上述转换。

为了方便比较，我们还需要将 "1G2T3M" 这种容量，全部转为 M 单位。具体解析策略可以是普通的字符串解析，或者正则解析。

Java算法源码

解法一：正则表达式

```
1 import java.util.*;
2 import java.util.regex.Matcher;
3 import java.util.regex.Pattern;
4
5 public class Main {
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8
9         int n = sc.nextInt();
10
11         String[] disks = new String[n];
12         for (int i = 0; i < n; i++) {
13             disks[i] = sc.next();
14         }
15
16         // 指定了自定义比较器后, Arrays.sort就是稳定排序
17         Arrays.sort(disks, (a, b) -> {
18             return Long.compare(calc(a), calc(b));
19         });
20
21         for (String disk : disks) {
22             System.out.println(disk);
23         }
24     }
25
26     public static Long calc(String disk) {
27         Matcher matcher = Pattern.compile("(\\d+)([MGT])").matcher(disk);
28
29         long ans = 0;
30
31         while (matcher.find()) {
32             long m = Long.parseLong(matcher.group(1));
33             String v = matcher.group(2);
34
35             switch (v) {
36                 case "M":
```

```

37         ans += m;38         break;
39     case "G":
40         ans += m * 1024;
41         break;
42     case "T":
43         ans += m * 1024 * 1024;
44         break;
45     }
46 }
47
48 return ans;
49 }
50 }
51

```

解法二：字符串解析

```

1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          int n = sc.nextInt();
8
9          String[] disks = new String[n];
10         for (int i = 0; i < n; i++) {
11             disks[i] = sc.next();
12         }
13
14         // 指定了自定义比较器后, Arrays.sort就是稳定排序
15         Arrays.sort(disks, (a, b) -> {
16             return Long.compare(calc(a), calc(b));
17         });
18
19         for (String disk : disks) {

```

```

20         System.out.println(disk);
21     }
22 }
23
24 public static Long calc(String disk) {
25     long ans = 0;
26
27     StringBuilder num = new StringBuilder();
28
29     for (int i = 0; i < disk.length(); i++) {
30         char c = disk.charAt(i);
31
32         if (Character.isDigit(c)) {
33             num.append(c);
34             continue;
35         }
36
37         long m = Long.parseLong(num.toString());
38         num = new StringBuilder();
39
40         switch (c) {
41             case 'M':
42                 ans += m;
43                 break;
44             case 'G':
45                 ans += m * 1024;
46                 break;
47             case 'T':
48                 ans += m * 1024 * 1024;
49                 break;
50         }
51     }
52
53     return ans;
54 }
55 }
56

```

JS算法源码

解法一：正则表达式

关于JS的正则表达式的exec方法：[RegExp.prototype.exec\(\) - JavaScript | MDN \(mozilla.org\)](#)

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6    const n = parseInt(await readline());
7
8    const disks = [];
9    for (let i = 0; i < n; i++) {
10      disks.push(await readline());
11    }
12
13    disks.sort((a, b) => calc(a) - calc(b));
14
15    for (let disk of disks) {
16      console.log(disk);
17    }
18  })();
19
20  function calc(disk) {
21    // 设置 g 标志, 后续循环调用 regexp.exec 时才会更新 regexp 的 lastIndex 属性（表示上次匹配的结束位置），否则不会更新 lastIndex 属性，导致死循环
22    const regexp = /(\d+)([MGT])/g;
23
24    let ans = 0;
25
26    let match;
27    while ((match = regexp.exec(disk))) {
28      const m = parseInt(match[1]);
```

```

29 |     const v = match[2];
30 |
31 |     switch (v) {
32 |         case "M":
33 |             ans += m;
34 |             break;
35 |         case "G":
36 |             ans += m * 1024;
37 |             break;
38 |         case "T":
39 |             ans += m * 1024 * 1024;
40 |             break;
41 |     }
42 | }
43 |
44 | return ans;
45 | }

```

解法二：字符串解析

```

1 | const rl = require("readline").createInterface({ input: process.stdin });
2 | var iter = rl[Symbol.asyncIterator]();
3 | const readline = async () => (await iter.next()).value;
4 |
5 | void (async function () {
6 |     const n = parseInt(await readline());
7 |
8 |     const disks = [];
9 |     for (let i = 0; i < n; i++) {
10 |         disks.push(await readline());
11 |     }
12 |
13 |     disks.sort((a, b) => calc(a) - calc(b));
14 |
15 |     for (let disk of disks) {

```



```
16 |     console.log(disk);17 | }
18 |})();
19 |
20 | function calc(disk) {
21 |     let ans = 0;
22 |
23 |     const num = [];
24 |
25 |     for (let c of disk) {
26 |         if (c >= "0" && c <= "9") {
27 |             num.push(c);
28 |             continue;
29 |         }
30 |
31 |         const m = parseInt(num.join(""));
32 |         num.length = 0;
33 |
34 |         switch (c) {
35 |             case "M":
36 |                 ans += m;
37 |                 break;
38 |             case "G":
39 |                 ans += m * 1024;
40 |                 break;
41 |             case "T":
42 |                 ans += m * 1024 * 1024;
43 |                 break;
44 |         }
45 |     }
46 |
47 |     return ans;
48 | }
```

解法一：正则表达式

```
1 import re
2
3 # 输入获取
4 n = int(input())
5 disks = [input() for _ in range(n)]
6
7
8 # 核心代码
9 def calc(s):
10     groups = re.compile(r'(\d+)([MGT])').findall(s)
11
12     ans = 0
13     for group in groups:
14         m = int(group[0])
15         v = group[1]
16
17         if v == 'M':
18             ans += m
19         elif v == 'G':
20             ans += m * 1024
21         else:
22             ans += m * 1024 * 1024
23
24     return ans
25
26
27 # 输出打印
28 if __name__ == '__main__':
29     disks.sort(key=lambda x: calc(x))
30
31     for disk in disks:
32         print(disk)
```

解法二：字符串解析

```
1  # 输入获取
2  n = int(input())
3  disks = [input() for _ in range(n)]
4
5
6  # 核心代码
7  def calc(s):
8      ans = 0
9
10     num = []
11     for c in s:
12         if c.isdigit():
13             num.append(c)
14             continue
15
16     m = int("".join(map(str, num)))
17     num.clear()
18
19     if c == 'M':
20         ans += m
21     elif c == 'G':
22         ans += m * 1024
23     else:
24         ans += m * 1024 * 1024
25
26     return ans
27
28
29 # 输出打印
30 if __name__ == '__main__':
31     disks.sort(key=lambda x: calc(x))
32
33     for disk in disks:
34         print(disk)
```

C算法源码

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4
5  #define MAX_LEN 30
6
7  typedef long long ll;
8
9  typedef struct Disk {
10     char disk[MAX_LEN];
11     ll capacity;
12     int index;
13 } Disk;
14
15 ll calc(char *disk) {
16     ll ans = 0;
17
18     char num[MAX_LEN] = {'\0'};
19
20     for (int i = 0; i < strlen(disk); i++) {
21         char c = disk[i];
22
23         if (c >= '0' && c <= '9') {
24             num[strlen(num)] = c;
25             continue;
26         }
27
28         ll m = atoll(num);
29         memset(num, '\0', MAX_LEN);
30
31         switch (c) {
32             case 'M':
33                 ans += m;
34                 break;
```

```

35         case 'G':
36             ans += m * 1024;
37         break;
38         case 'T':
39             ans += m * 1024 * 1024;
40             break;
41     }
42 }
43
44 return ans;
45 }
46
47 int cmp(const void *a, const void *b) {
48     Disk *A = (Disk *) a;
49     Disk *B = (Disk *) b;
50
51     if (A->capacity != B->capacity) {
52         // 注意下面差值可能非常大（超出int），此时直接返回差值的话，会整型溢出，导致错误比较结果
53         return A->capacity > B->capacity ? 1 : -1;
54     } else {
55         // 稳定排序
56         return A->index - B->index;
57     }
58 }
59
60 int main() {
61     int n;
62     scanf("%d", &n);
63
64     Disk disks[n];
65     for (int i = 0; i < n; i++) {
66         scanf("%s", disks[i].disk);
67         disks[i].capacity = calc(disks[i].disk);
68         disks[i].index = i;
69     }
70
71     qsort(disks, n, sizeof(disks[0]), cmp);

```

```
72 | 73 |     for (int i = 0; i < n; i++) {
74 |     puts(disks[i].disk);
75 |     }
76 |
77 |     return 0;
78 | }
```

C++算法源码

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  long long calc(string &s) {
6      long long ans = 0;
7
8      string num;
9      for (const auto &c: s) {
10         if (isdigit(c)) {
11             num += c;
12             continue;
13         }
14
15         long long m = stoll(num);
16         num.clear();
17
18         switch (c) {
19             case 'M':
20                 ans += m;
21                 break;
22             case 'G':
23                 ans += m * 1024;
24                 break;
25             case 'T':
```

```

26         ans += m * 1024 * 1024;27         break;
28     }
29 }
30
31 return ans;
32 }
33
34 int main() {
35     int n;
36     cin >> n;
37
38     vector<string> disks(n);
39     for (int i = 0; i < n; i++) {
40         cin >> disks[i];
41     }
42
43     stable_sort(disks.begin(), disks.end(), [](string a, string b) {
44         return calc(a) < calc(b);
45     });
46
47     for (const auto &disk: disks) {
48         cout << disk << endl;
49     }
50
51     return 0;
52 }

```